

Pratique de la Data Science

Cours 3 : Deep Learning

Université Paris Dauphine – Master 2 Ingénierie Statistique et Financière (ISF)

Pierre Fihey – *pierre.fihey@ip-paris.fr*

Plan du cours

I. Introduction aux réseaux de neurones

- Perceptron
- Couches cachées et fonctions d'activations
- Descente de gradient

II. Multi Layer Perceptron (MLP)

- Initialisation des poids
- Régularisation
- Normalisation

III. Recurrent Neural Network (RNN)

- RNN
- LSTM

Plan du cours

I. Introduction aux réseaux de neurones

- Perceptron
- Couches cachées et fonctions d'activations
- Descente de gradient

II. Multi Layer Perceptron (MLP)

- Initialisation des poids
- Régularisation
- Normalisation

III. Recurrent Neural Network (RNN)

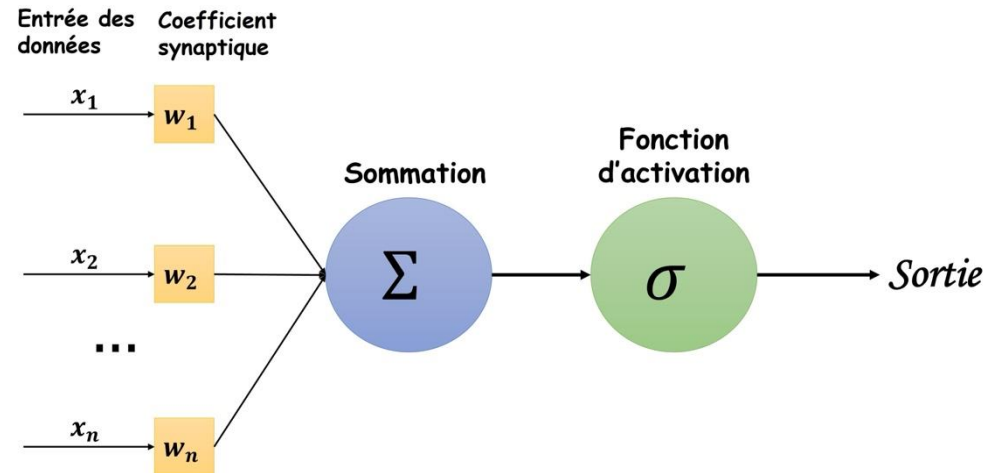
- RNN
- LSTM

Perceptron : Premier modèle de neurone artificiel

Modèle perceptron :

- **Tâche** : Classification binaire ($\mathcal{Y} \in \{0, 1\}$).
- **Formule** : $\hat{y} = \sigma(x^T w + b)$ avec σ la fonction seuil :

$$\sigma(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{sinon} \end{cases}$$



Modèle perceptron (Rosenblatt. 1957)

Perceptron : Premier modèle de neurone artificiel

Entraînement du modèle :

- **Mise à jour des poids** (uniquement si $\hat{y} \neq y$) :

$$\begin{cases} w_i \leftarrow w_i + \alpha(y - \hat{y})x_i \\ b \leftarrow b + \alpha(y - \hat{y}) \end{cases}$$

- Fin si convergence ou nombre maximum d'itérations atteint.

Limitations :

- Optimisation binaire, instable et peut ne pas converger.
- **Un seul neurone** $\Rightarrow$ Simple combinaison linéaire des inputs
 $\Rightarrow$ Ne peut résoudre que des problèmes linéairement séparables.

Plan du cours

I. Introduction aux réseaux de neurones

- Perceptron
- **Couches cachées et fonctions d'activations**
- Descente de gradient

II. Multi Layer Perceptron (MLP)

- Initialisation des poids
- Régularisation
- Normalisation

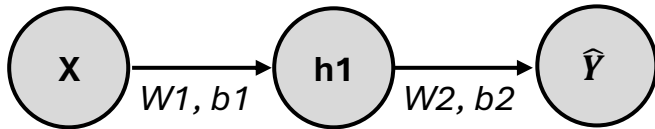
III. Recurrent Neural Network (RNN)

- RNN
- LSTM

Réseaux profonds et fonctions d'activations

Objectif : Introduire la non-linéarité

1. Ajout de couches cachées :



2. Ajout de fonctions d'activations :

- On considère un réseau de neurones à deux couches et un input x . Sans fonction d'activation, on a :

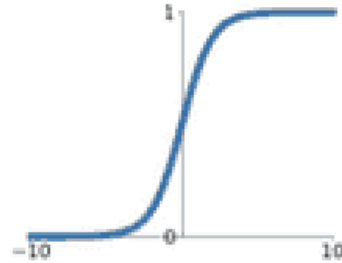
$$\begin{cases} h_1 = W^1 x + b^1 \\ \hat{y} = W^2 h_1 + b^2 \end{cases} \Rightarrow \hat{y} = (W^2 W^1) x + (W^2 b^1 + b^2)$$

- Donc **$\hat{y}$ combinaison linéaire des inputs**, ne permet d'approximer que des fonctions linéaires.
- L'ajout de fonction d'activation non linéaire σ sur la première couche ($h = \sigma(W^1 x + b^1)$) permet de **casser cette linéarité**.

Fonctions d'activations : « Casser la linéarité »

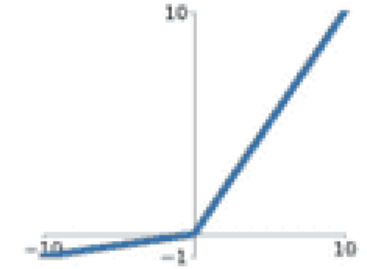
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



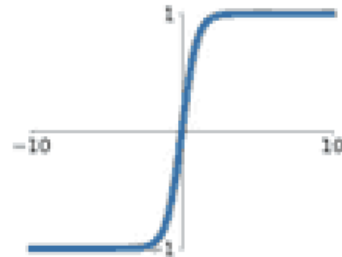
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

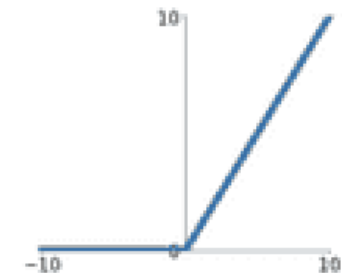


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

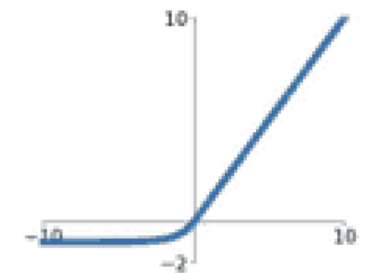
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Réseaux profonds et fonctions d'activations

Fonction d'activation	Expression mathématique	Intérêt	Limites	Cas d'usage
Sigmoïde	$\sigma(x) = \frac{1}{1+e^{-x}}$	Transforme la sortie en probabilité (0 à 1).	Vanishing gradient pour les grandes valeurs de $ x $.	Dernière couche pour classification binaire.
Tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	Centrée sur zéro $\implies$ permet une meilleure convergence.	Vanishing gradient similaire à Sigmoïde.	Réseaux récurrents (RNN), NLP.
ReLU	$ReLU(x) = \max(0, x)$	Simple, évite le vanishing gradient pour $x > 0$.	Neurones morts (sortie toujours 0 si $x < 0$).	Couches cachées de réseaux profonds, CNN.
Leaky ReLU	$LeakyReLU(x) = \max(0.01x, x)$	Permet un petit gradient même pour $x < 0$, évite les neurones morts.	Nécessite un paramètre manuel (α).	Alternative à ReLU quand les neurones morts posent problème.
Softmax	$Softmax(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$	Convertit un vecteur en distribution de probabilités.	Sensible aux valeurs extrêmes, peut saturer.	Dernière couche pour classification multi-classes.

Principales fonctions d'activation

Théorème d'approximation universelle :

- Toute fonction continue définie sur un ensemble compact peut être approchée d'aussi près que l'on veut par un réseau neuronal à une couche cachée avec activation sigmoïde.

Plan du cours

I. Introduction aux réseaux de neurones

- Perceptron
- Couches cachées et fonctions d'activations
- **Descente de gradient**

II. Multi Layer Perceptron (MLP)

- Initialisation des poids
- Régularisation
- Normalisation

III. Recurrent Neural Network (RNN)

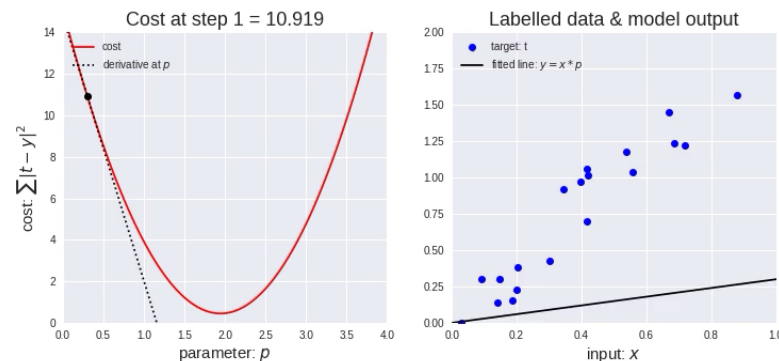
- RNN
- LSTM

Rappel : Descente de Gradient

Fonctions d'activations différentiable $\Rightarrow$ Rend possible l'apprentissage par descente de gradient.

Algorithme de descente de gradient :

1. **Initialisation** des poids (aléatoire ou zéro)
2. Répéter jusqu'à convergence :
 1. **Prédiction** du modèle : $\hat{y} = \sigma(\theta x)$
 2. Calcul de l'**erreur** $L(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{y}_i)$
 3. Calcul du **gradient de la fonction de coût** $\nabla L(\theta) = \frac{\partial L}{\partial \theta}$
 4. **Mise à jour des poids** (direction opposée au gradient) $\theta \leftarrow \theta - \alpha \nabla L(\theta)$
 5. **Critère d'arrêt** : $\max \text{itérations} |\nabla L(\theta)| \approx 0, \dots$



Stochastic Gradient Descent

Problématique : Coût computationnel de la descente de gradient (prohibitif sur larges datasets).

Stochastic Gradient Descent :

- Plutôt que de calculer le gradient sur l'ensemble du dataset, on ne le calcule que sur un échantillon à chaque itération.

$$\theta_{t+1} = \theta_t - \alpha \nabla L(\theta_t, x_i, y_i)$$

- Échantillons choisis aléatoirement $\Rightarrow$ Peut permettre d'éviter des minima locaux.

Avantages : Entraînement moins coûteux et meilleure exploration des solutions possibles.

Limites : Convergence bruitée et instable

Mini-batch Gradient Descent

Mini-batch Gradient Descent :

- Compromis entre descente de gradient classique et SGD, les gradients sont calculés sur des mini-batches à chaque itération.

$$\theta_{t+1} = \theta_t - \alpha \nabla L(\theta_t, x_{[i:i+m]}, y_{[i:i+m]})$$

Avantages : Compromis entre vitesse de calcul et stabilité. Possibilité de calculs parallèles sur GPU.

- Nécessite un choix optimal de batch_size

Momentum

Objectif : Accélérer la convergence en ajoutant un principe d'inertie dans la mise à jour des poids.

Momentum :

- On conserve une moyenne mobile des gradients précédents pour l'ajouter au gradient actuel :

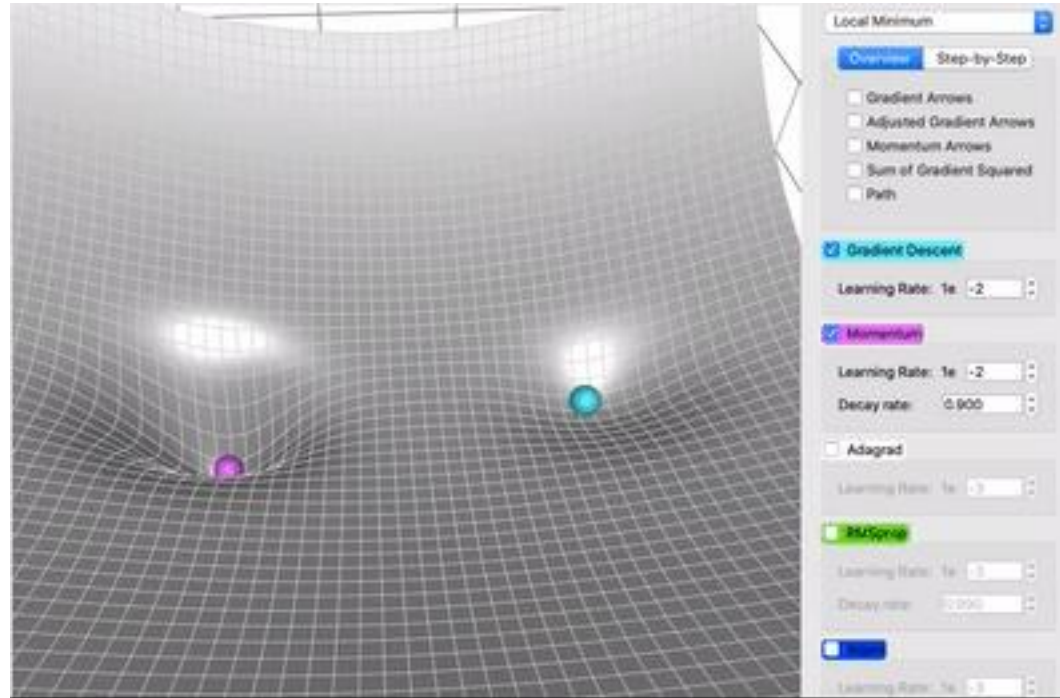
$$\begin{cases} v_t = \beta v_{t-1} + (1 - \beta) \nabla J(\theta_t) \\ \theta_{t+1} = \theta_t - \alpha v_t \end{cases}$$

- Permet d'**accélérer la descente** dans les dimensions où les gradients sont cohérents et de **réduire les oscillations** dans les dimensions où le gradient varie fortement.

Avantages : Meilleure stabilité, convergence rapide, évite les minima locaux.

- Nécessite un choix optimal de β .

Momentum



AdaGrad

Objectif : Adapter les learning rate dynamiquement pour chaque feature.

AdaGrad :

- Pour chaque feature, on calcule l'accumulation des gradients précédents afin d'adapter le learning rate. :

$$\begin{cases} G_{t,j} = G_{t-1,j} + \nabla J(\theta_{t,j})^2 \\ \theta_{t+1} = \theta_{t,j} - \frac{\alpha}{\sqrt{G_{t,j} + \epsilon}} \nabla J(\theta_{t,j}) \end{cases}$$

Permet de diminuer le learning rate pour les features qui ont été fortement mises à jour tout en conservant celui des features plus rares.

Adam

Objectif : Combiner les avantages de Momentum et AdaGrad pour accélérer la convergence tout en adaptant dynamiquement le learning rate.

Adam :

- Par rapport à AdaGrad, on ajoute une pondération sur l'accumulation des carrés des gradients pour oublier progressivement les plus anciens :

$$\begin{cases} v_t = \beta_1 v_{t-1} + (1 - \beta_1) \nabla J(\theta_t) \\ G_{t,j} = \beta_2 G_{t-1,j} + (1 - \beta_2) \nabla J(\theta_{t,j})^2 \\ \theta_{t+1,j} = \theta_{t,j} - \alpha \frac{v_t}{\sqrt{G_{t,j} + \epsilon}} \end{cases}$$

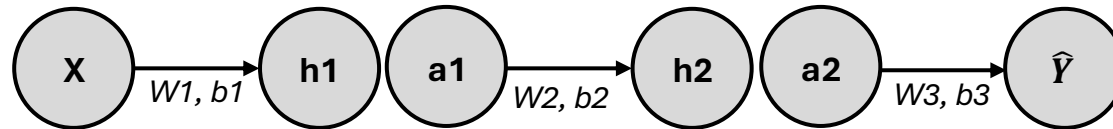
Adam est l'optimiseur le plus utilisé en Deep Learning.

Rappel : Backpropagation

Objectif : Calculer le gradient de la Loss par rapport à chaque poids du réseau.

Principe : »Chain Rule « pour propager l'erreur jusqu'à l'entrée. Les gradients de chaque couche sont stockés pour faciliter le calcul du gradient de la couche précédente.

- On considère le réseau suivant ($h_i = w_i a_{i-1} + b_i$, $a_i = \sigma(h_i)$)



Calcul des gradients (Chain Rule):

- $\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial h_i} \cdot a_{i-1}$
- $\frac{\partial \mathcal{L}}{\partial h_i} = \frac{\partial \mathcal{L}}{\partial a_i} \cdot \sigma'(h_i)$
- $\frac{\partial \mathcal{L}}{\partial a_i} = \frac{\partial \mathcal{L}}{\partial h_{i+1}} \cdot w_{i+1}$

Plan du cours

I. Introduction aux réseaux de neurones

- Perceptron
- Couches cachées et fonctions d'activations
- Descente de gradient

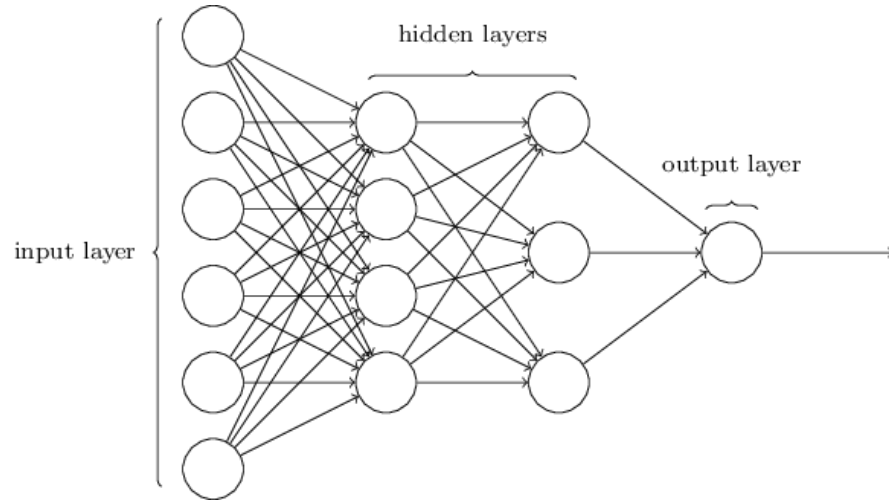
II. Multi Layer Perceptron (MLP)

- Initialisation des poids
- Régularisation
- Normalisation

III. Recurrent Neural Network (RNN)

- RNN
- LSTM

Multi Layer Perceptron (MLP)



Multi Layer Perceptron :

- Chaque neurone est connecté à tous les neurones de la couche précédente (**Fully Connected Layers**).

$$z = f(\sum_{i=1}^{n_{in}} w_i x_i + b)$$

Couche de sortie :

- **Régression** : 1 neurone → **Activation linéaire**
- **Classification binaire** : 1 neurone → **Activation sigmoïde**
- **Classification multiclasse** : K neurones → **Activations softmax**

Limites :

- Nombre élevé de paramètres : Risque d'overfitting et coût computationnel élevé.
- Peu efficace pour des données séquentielles (Séries temporelles, texte....)

Initialisation des poids

Entraînement du réseau très sensible à l'initialisation des poids :

- Output d'un neurone : $z = f(\sum_{i=1}^{n_{in}} w_i x_i + b)$
- **Exploding Gradient** : Si w_i et n_i très grands $\Rightarrow Z$ devient trop grand (gradient aussi lors de la backpropagation).
- **Vanishing Gradient** : Si w_i et n_i très petits $\Rightarrow Z$ devient trop petit (gradient aussi lors de la backpropagation).

Initialisation de Xavier-Glorot :

- Equilibre la variance des poids en fonction du nombre d'entrée et de sorties de chaque neurone :

$$Var[w] = \frac{1}{n_{in} + n_{out}} \Rightarrow W \sim \mathcal{U}\left(-\frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}, \frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}\right)$$

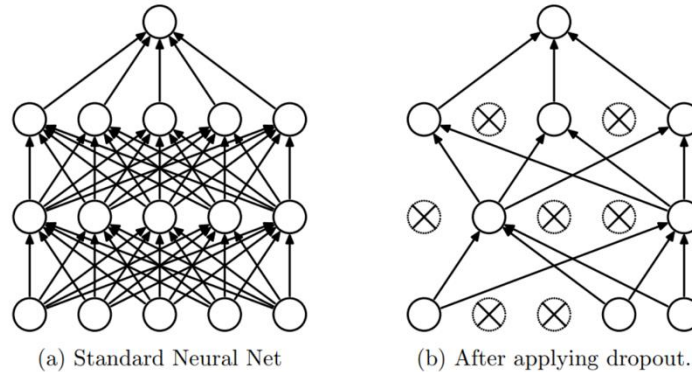
- Permet de stabiliser les flux entre les couches et d'éviter l'explosion ou la disparition des gradients.

Regularisation

Nombre très élevé de paramètre dans le réseau de neurones $\Rightarrow$ **Risque d'overfitting.**

Dropout :

- Durant l'entraînement, à chaque itération : Chaque neurone a une probabilité p d'être désactivé.

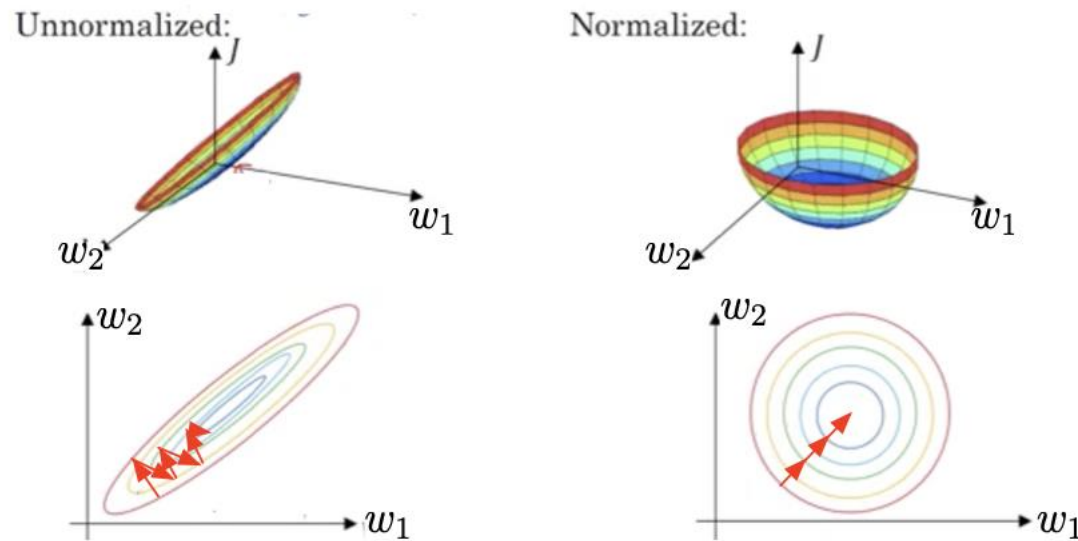


Effets du dropout :

- Evite la co-adaptation entre les neurones.
- Evite l'overfitting et permet une meilleure généralisation.
- Réseau plus petit $\Rightarrow$ Plus simple à entraîner.

Inputs normalisation

Features avec échelles très différentes : Les poids sont très déséquilibrés (certaines variables auront plus d'influence) et l'optimisation oscille fréquemment.



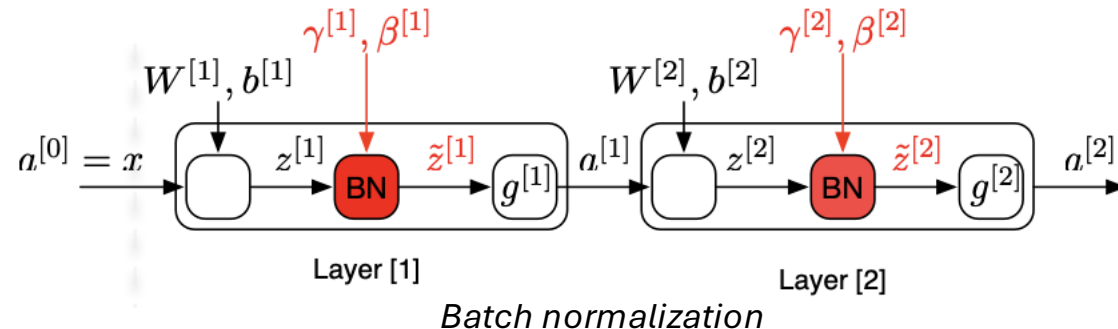
Après normalisation : Les poids ont des valeurs proches $\Rightarrow$ Gradients équilibrés et apprentissage stable.

Batch normalization

Internal covariate shift:

- Les inputs d'une couche $[L]$ sont les outputs de la couche précédente $[L-1]$.
- Ces inputs dépendent des paramètres de $[L-1]$ qui **évoluent à chaque itération**.
- La couche $[L]$ doit donc réajuster ses poids pour s'adapter, ce qui rend l'**apprentissage plus lent et instable**.

Batch normalization : On normalise les outputs de la couche précédente avant l'activation.



Batch normalization

Problème : Chaque neurone doit apprendre des caractéristiques différentes, renvoyer des signaux différents.

⇒ On ne veut pas que tous les neurones aient une moyenne de 0 et une variance de 1.

Solution : Après normalisation des activations $z(i)$, on leur applique une transformation affine :

⇒ $\hat{\mathbf{z}}^{(i)} = \boldsymbol{\gamma} \mathbf{z}^{(i)} + \boldsymbol{\beta}$ avec $\boldsymbol{\gamma}$ et $\boldsymbol{\beta}$ appris pendant l'entraînement.

- Permet de conserver la distribution de chaque activation sans forcer une unique distribution.

Plan du cours

I. Introduction aux réseaux de neurones

- Perceptron
- Couches cachées et fonctions d'activations
- Descente de gradient

II. Multi Layer Perceptron (MLP)

- Initialisation des poids
- Régularisation
- Normalisation

III. Recurrent Neural Network (RNN)

- RNN
- LSTM

Données séquentielles

Définition : Types de données pour lesquelles l'ordre des éléments est important.

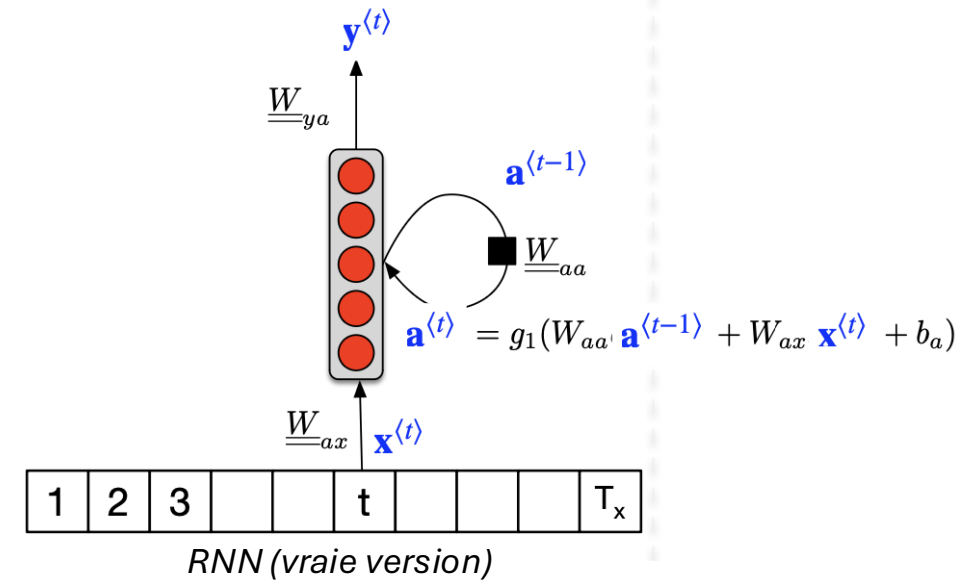
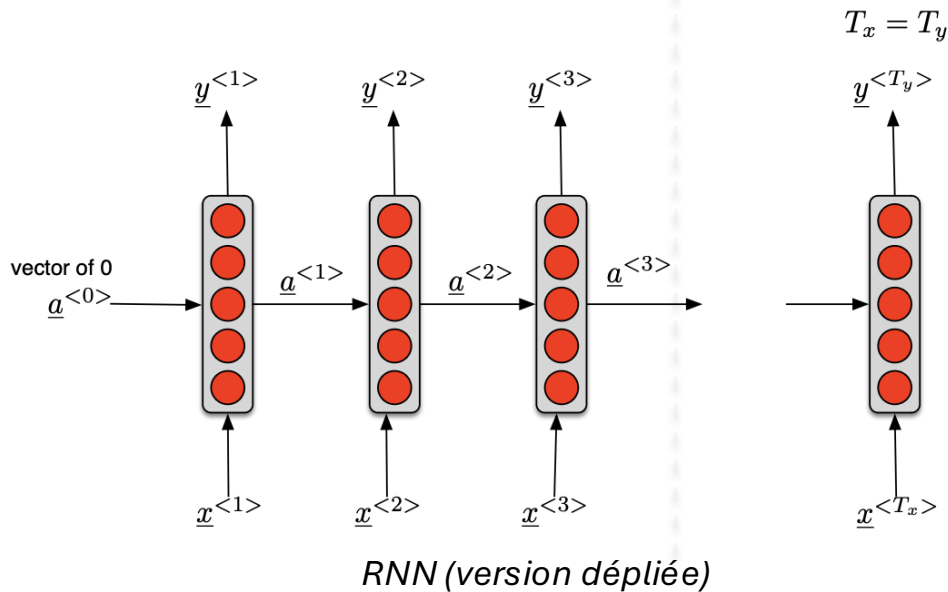
Exemple :

- **Séries temporelles :** Stocks market, Audios, Vidéos...
- **Données ordonnées :** Texte, ADN ...

MLP classiques traitent chaque donnée de manière indépendante.

⇒ Impossible d'exploiter ces relations temporelles.

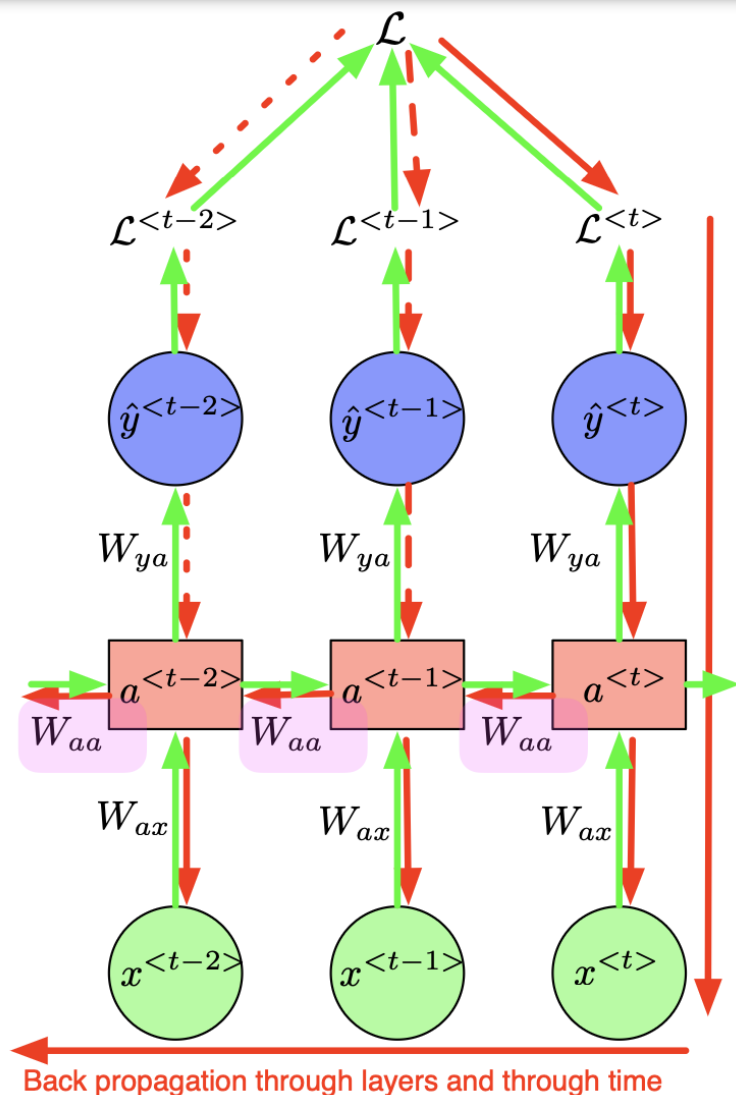
Recurrent Neural Network (RNN)



Paramètres (les mêmes à chaque étape t) :

- $\underline{W}_{a<x>} : \underline{x}^{<t>} \rightarrow \underline{a}^{<t>}$
- $\underline{W}_{aa} : \underline{a}^{<t-1>} \rightarrow \underline{a}^{<t>}$
- $\underline{W}_{ya} : \underline{a}^{<t>} \rightarrow \underline{y}^{<t>}$

Backpropagation Through Time (BPTT)



Formule :

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial W_{aa}} = \sum_{t=1}^T \frac{\partial \mathcal{L}^{<t>}}{\partial W_{aa}} \\ \frac{\partial \mathcal{L}^{<t>}}{\partial W_{aa}} = \sum_{k=0}^t \left(\prod_{i=k+1}^t \frac{\partial a^{<i>}}{\partial a^{<i-1>}} \right) \frac{\partial a^{<k>}}{\partial W_{aa}} \end{cases}$$

Avec $\left(\prod_{i=k+1}^t \frac{\partial a^{<i>}}{\partial a^{<i-1>}} \right) \frac{\partial a^{<k>}}{\partial W_{aa}}$ la contribution de l'état (temps) $<k>$ au gradient de la loss au temps $<t>$.

BPTT = Exploding et Vanishing gradient

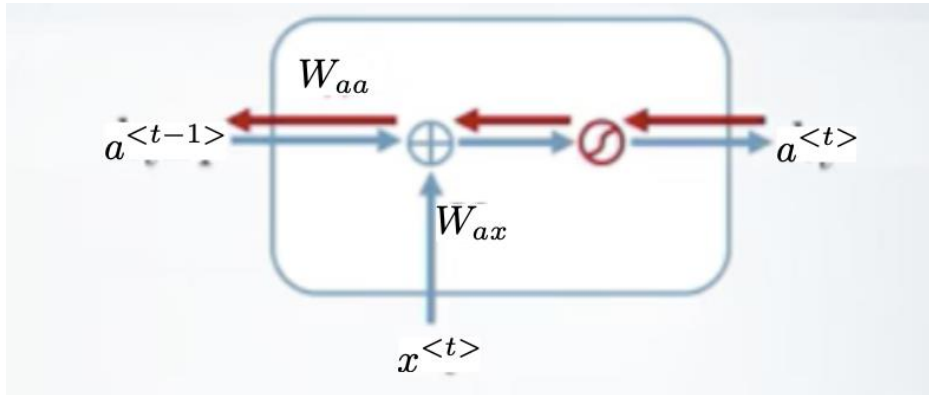
$$\frac{\partial \mathcal{L}^{(t)}}{\partial W_{aa}} = \sum_{k=0}^t \left(\prod_{i=k+1}^t \frac{\partial a^{(i)}}{\partial a^{(i-1)}} \right) \frac{\partial a^{(k)}}{\partial W_{aa}}$$

Vanishing/Exploding gradient :

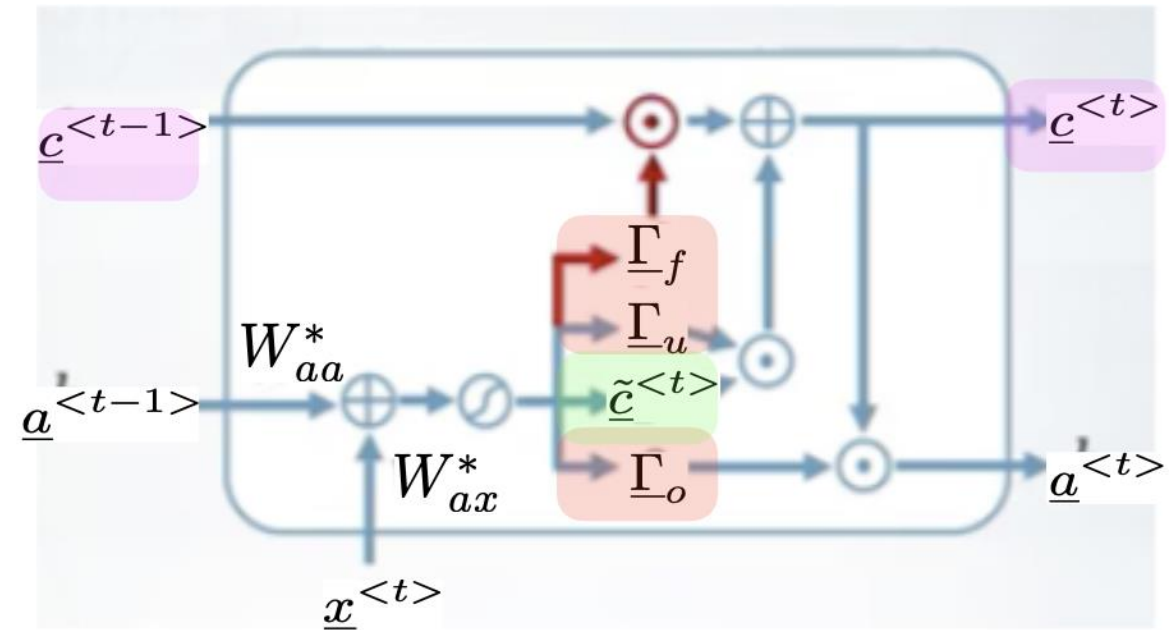
- **Vanishing gradient** : Si $\left| \frac{\partial a^{(i)}}{\partial a^{(i-1)}} \right| < 1 \Rightarrow$ Le gradient tend rapidement vers 0 $\Rightarrow$ Les états lointains n'ont plus aucune contribution pour l'entraînement.
- **Exploding gradient** : Si $\left| \frac{\partial a^{(i)}}{\partial a^{(i-1)}} \right| > 1 \Rightarrow$ Le gradient tend rapidement vers $\infty \Rightarrow$ Apprentissage instable (gradient peut devenir NaN).

Limite des RNNs classiques : Traiter des longues séquences.

Gated units : Long-Short Term Memory (LSTM)



RNN classique

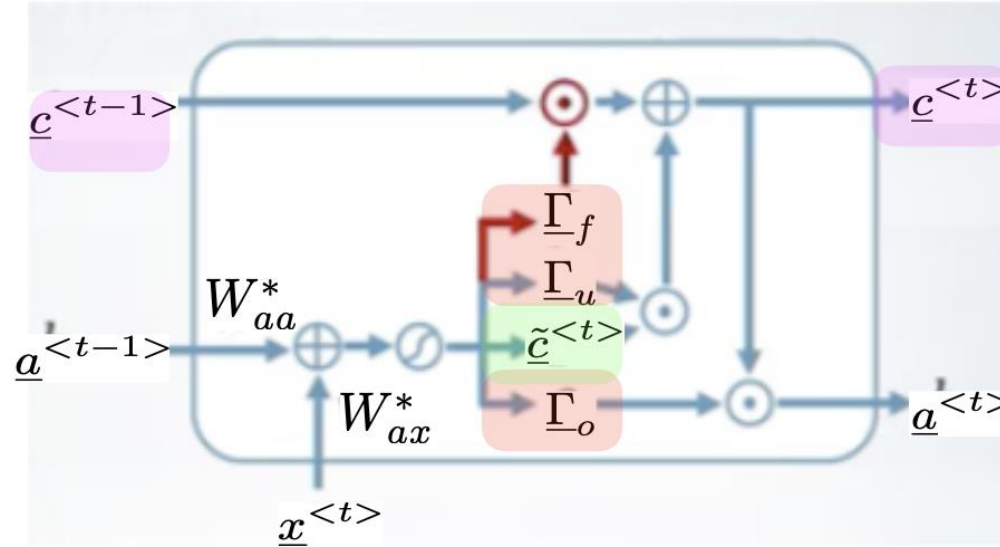


LSTM

Cellule LSTM :

- Objectif : Conserver en mémoire les anciennes informations de manière stable et contrôlée.
 $\Rightarrow a^{<t>}$ est affecté par la mémoire à court terme (via Γ_o) et à long terme (via $c^{<t>}$).

Gated units : Long-Short Term Memory (LSTM)



Gates :

- **Forget Gate (Γ_F) :** Quelles informations de $c^{<t-1>}$ sont oubliées.

$$\Rightarrow \Gamma_F = \sigma(W_{ax}^f x^{<t>} + W_{aa}^f a^{<t-1>} + b^f)$$

- **Update Gate (Γ_U) :** Quelles informations sont ajoutées à $c^{<t-1>}$.

$$\Rightarrow \Gamma_U = \sigma(W_{ax}^U x^{<t>} + W_{aa}^U a^{<t-1>} + b^U)$$

- **Output Gate (Γ_O) :** Quelle partie de $c^{<t>}$ est utilisée pour générer $a^{<t>}$.

$$\Rightarrow \Gamma_O = \sigma(W_{ax}^O x^{<t>} + W_{aa}^O a^{<t-1>} + b^O)$$